

PYTHON WITH DJANGO

Week 2 - BEAUTIFUL SOUP

AIM: Write a program using BeautifulSoup4 library for web scraping for a given URL

CODE:

[ROUTE.PY](#)

```
from question import app
from flask import render_template
from question.scrapers import scrape_website

@app.route('/')
def home():
    # Use BeautifulSoup to scrape quotes
    result = scrape_website("http://quotes.toscrape.com")
    return render_template('index.html', result=result)
```

[INDEX.HTML](#)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>BeautifulSoup Scraper</title>
  </head>
  <body>
    <h1>Scraped Quotes</h1>
```

```
<ul>
    {% for quote in result %}
    <li>{{ quote }}</li>
    {% endfor %}
</ul>
</body>
</html>
```

SCRAPPER.PY

```
from bs4 import BeautifulSoup
import requests
```

```
def scrape_website(url):
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")

    # Scrape quotes from the page
    quotes = soup.find_all("span", class_="text")
    authors = soup.find_all("small", class_="author")

    # Prepare a list of quotes and authors
    result = []

    for quote, author in zip(quotes, authors):
        result.append(f"\{quote.text}\n - {author.text}")

    return result
```

OUTPUT:

Scraped Quotes

- "“The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.”” - Albert Einstein
- "“It is our choices, Harry, that show what we truly are, far more than our abilities.”” - J.K. Rowling
- "“There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle.”” - Albert Einstein
- "“The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid.”” - Jane Austen
- "“Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring.”” - Marilyn Monroe
- "“Try not to become a man of success. Rather become a man of value.”” - Albert Einstein
- "“It is better to be hated for what you are than to be loved for what you are not.”” - André Gide
- "“I have not failed. I've just found 10,000 ways that won't work.”” - Thomas A. Edison
- "“A woman is like a tea bag; you never know how strong it is until it's in hot water.”” - Eleanor Roosevelt
- "“A day without sunshine is like, you know, night.”” - Steve Martin

AIM: Develop a sample Hello World page using Flask framework

CODE :

```
from flask import render_template
```

```
def home():
```

```
    return "Hello, world!"
```

```
init.py
```

```
from flask import Flask
```

```
def create_app():
```

```
    app = Flask(__name__)
```

```
    # Register routes (in routes.py)
```

```
    from .routes import home
```

```
    app.add_url_rule('/', 'home', home)
```

```
    return app
```

OUTPUT:

HELLO,WORLD!

WEEK 3 - DJANGO

AIM: Create a Sample “Hello World” Application using Django

CODE:

[views.py](#)

```
from django.http import HttpResponse
```

```
def hello_world(request):  
    return HttpResponse("Hello, World!")
```

[urls.py](#)

```
from django.contrib import admin  
from django.urls import path  
from .views import hello_world # import the view  
  
urlpatterns = [  
    path('admin/', admin.site.urls),  
    path('', hello_world), # root URL shows Hello World  
]
```

OUTPUT:

Hello, World !

WEEK 4 - DJANGO

1.

AIM: Create a Login and Registration Page using MVC architecture in Django Framework

forms.py

```
from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import AuthenticationForm

class RegisterForm(forms.ModelForm):
    password = forms.CharField(widget=forms.PasswordInput)
    confirm_password = forms.CharField(widget=forms.PasswordInput)

    class Meta:
        model = User
        fields = ['username', 'email', 'password']

    def clean(self):
        cleaned_data = super().clean()
        password = cleaned_data.get("password")
        confirm_password = cleaned_data.get("confirm_password")
        if password != confirm_password:
            raise forms.ValidationError("Passwords do not match.")

class LoginForm(AuthenticationForm):
    username = forms.CharField()
    password = forms.CharField(widget=forms.PasswordInput)
```

views.py

```
from django.shortcuts import render, redirect
```

```
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from .forms import RegisterForm, LoginForm
```

```
def register_view(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)
            user.set_password(form.cleaned_data['password'])
            user.save()
            login(request, user)
            return redirect('home')
    else:
        form = RegisterForm()
    return render(request, 'register.html', {'form': form})
```

```
def login_view(request):
    if request.method == 'POST':
        form = LoginForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect('home')
    else:
        form = LoginForm()
    return render(request, 'login.html', {'form': form})
```

```
def logout_view(request):
    logout(request)
    return redirect('login')
```

```
@login_required
def home_view(request):
    return render(request, 'home.html', {'user': request.user})
```

urls.py

```
from django.contrib import admin
from django.urls import path
from .views import register_view, login_view, logout_view, home_view

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home_view, name='home'),
    path('register/', register_view, name='register'),
    path('login/', login_view, name='login'),
    path('logout/', logout_view, name='logout'),
]
```

settings.py [add this to end of the file]

```
LOGIN_URL = '/login/'
LOGOUT_REDIRECT_URL = '/login/'

TEMPLATES[0]['DIRS'] = [BASE_DIR / "question" / "templates"]
```

Templates

register.html

```
<h2>Register</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Register</button>
</form>
<a href="{% url 'login' %}">Already have an account?</a>
```

home.html

```
<h2>Welcome, {{ user.username }}!</h2>
<a href="{% url 'logout' %}">Logout</a>
```


login.html

```
<h2>Login</h2>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Login</button>
</form>
<a href="{% url 'register' %}">Create an account</a>
```

OUTPUT :

Login

Username:

Password:

Login

[Create an account](#)

Register

Username: Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Email address:

Password:

Confirm password:

Register

[Already have an account?](#)

2.

AIM: Create a sample page in Django by integrating Bootstrap.

urls.py

```
from django.contrib import admin
from django.urls import path
from django.shortcuts import render

def home(request):
    return render(request, 'index.html')

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", home), # this will serve your Bootstrap page
]
```

Settings.py

```
'DIRS': [BASE_DIR / "question/templates"],
```

Templates

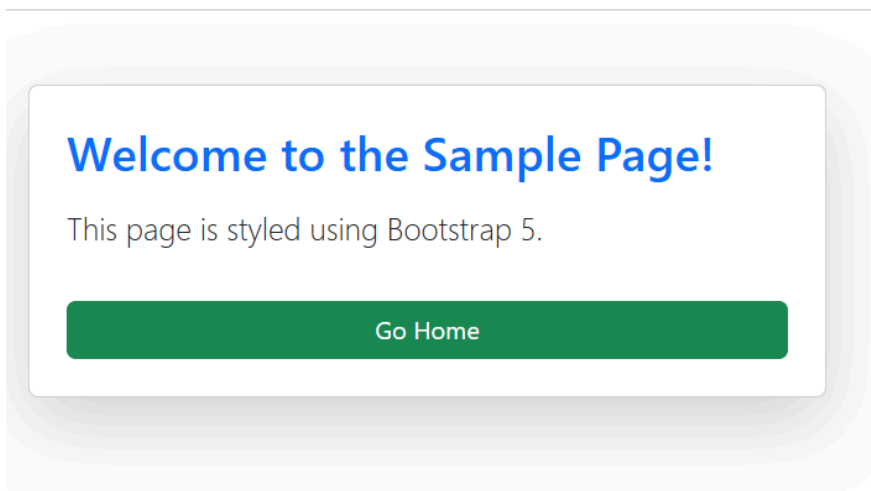
index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sample Page</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
```

```
<div class="container mt-5">
  <div class="card shadow-lg p-4 rounded">
    <h1 class="text-primary mb-3">Welcome to the Sample Page!</h1>
    <p class="lead">This page is styled using Bootstrap 5.</p>
    <a href="/" class="btn btn-success mt-3">Go Home</a>
  </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>
```

OUTPUT :



WEEK 5 - DJANGO

1.

AIM: Create an application with Tables, grids in Django

urls.py

```
from django.contrib import admin
from django.urls import path
from django.shortcuts import render

# Dummy data
students = [
    {'id': 1, 'name': 'Alice', 'branch': 'CSE', 'score': 89},
    {'id': 2, 'name': 'Bob', 'branch': 'ECE', 'score': 75},
    {'id': 3, 'name': 'Charlie', 'branch': 'MECH', 'score': 92},
    {'id': 4, 'name': 'Diana', 'branch': 'CIVIL', 'score': 81},
]

def table_view(request):
    return render(request, 'tables.html', {'students': students})

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', table_view), ]
```

Settings.py

```
'DIRS': [BASE_DIR / 'question' / 'templates'],
```

```
# or 'DIRS':['your_app/templates']
```

templates/tables.html

```
<!-- question/templates/table.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Table Grid in Django</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <div class="container mt-5">
    <h2 class="mb-4 text-center">Student Records</h2>
    <table class="table table-bordered table-striped">
      <thead class="table-dark">
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Branch</th>
          <th>Score</th>
        </tr>
      </thead>
      <tbody>
        {% for student in students %}
        <tr>
          <td>{{ student.id }}</td>
```

```
<td>{{ student.name }}</td>
<td>{{ student.branch }}</td>
<td>{{ student.score }}</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</body>

</html>
```

OUTPUT:

Student Records

ID	Name	Branch	Score
1	Alice	CSE	89
2	Bob	ECE	75
3	Charlie	MECH	92
4	Diana	CIVIL	81

2.

AIM: Create a Django App with Carousels feature.

CODE:

[question/static/images/](#)

- Keep images downloads in this folder

[question/templates/carousel.html](#)

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>Carousel</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
<div class="container mt-5">
  <div id="myCarousel" class="carousel slide" data-bs-ride="carousel">
    <div class="carousel-inner">
      {% for img in images %}
        <div class="carousel-item {% if forloop.first %}active{% endif %}">
          
        </div>
      {% endfor %}
    </div>
    <button class="carousel-control-prev" type="button" data-bs-target="#myCarousel"
data-bs-slide="prev">
      <span class="carousel-control-prev-icon"></span>
```

```

        </button>

        <button class="carousel-control-next" type="button" data-bs-target="#myCarousel"
data-bs-slide="next">

            <span class="carousel-control-next-icon"></span>

        </button>
    </div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

settings.py

```

STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'question' / 'static']

```

```

'DIRS': [BASE_DIR / 'question' / 'templates'],

```

urls.py

```

from django.contrib import admin
from django.urls import path
from .views import carousel_view
from django.conf import settings
from django.conf.urls.static import static

```

```

urlpatterns = [
    path('admin/', admin.site.urls),

```



```
    path('', carousel_view, name='carousel'),  
]
```

```
if settings.DEBUG:
```

```
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATICFILES_DIRS[0])
```

views.py

```
from django.shortcuts import render
```

```
from django.conf import settings
```

```
import os
```

```
def carousel_view(request):
```

```
    image_dir = os.path.join(settings.BASE_DIR, 'question', 'static', 'images')
```

```
    try:
```

```
        image_list = os.listdir(image_dir)
```

```
        image_urls = [f'images/{img}' for img in image_list if img.lower().endswith(('.jpg', '.jpeg',  
'png'))]
```

```
    except FileNotFoundError:
```

```
        image_urls = []
```

```
    return render(request, 'carousel.html', {'images': image_urls})
```

settings.py

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [BASE_DIR / 'question' / 'static']
```

'DIRS': [BASE_DIR / 'question' / 'templates'],

OUTPUT:



WEEK 6 - AUTHENTICATION AND MAIL

1.

AIM: Create a registration page using Authentication System using Django.

forms.py

```
# question/forms.py
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class RegisterForm(UserCreationForm):
    email = forms.EmailField()

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']
```

views.py

```
# question/views.py
from django.shortcuts import render, redirect
from .forms import RegisterForm

def register_view(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            form.save()
```

```

        return redirect('login') # we'll define this route later
    else:
        form = RegisterForm()
    return render(request, 'register.html', {'form': form})

```

Urls.py

```

from django.contrib import admin
from django.urls import path
from django.views.generic import RedirectView
from .views import register_view
from django.contrib.auth import views as auth_views

urlpatterns = [
    path('', RedirectView.as_view(url='login/', permanent=False)),    path('register/', register_view,
name='register'),
    path('login/', auth_views.LoginView.as_view(template_name='login.html'), name='login'),
    path('logout/', auth_views.LogoutView.as_view(next_page='login'), name='logout'),
    path('admin/', admin.site.urls),
]

```

questions/templates/register.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registration</title>
    <script src="https://cdn.tailwindcss.com"></script>

```

```

</head>

<body class="bg-gray-100 h-screen flex items-center justify-center">

  <div class="bg-white p-8 rounded-lg shadow-lg w-full max-w-md">
    <h2 class="text-3xl font-semibold text-center mb-6">Create an Account</h2>

    <form method="POST" action="{% url 'register' %}" class="space-y-4">
      {% csrf_token %}
      <div>
        <label for="username" class="block text-sm font-medium text-gray-700">Username</label>
        <input type="text" id="username" name="username" required class="w-full px-4 py-2 mt-2 border rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500">
      </div>

      <div>
        <label for="email" class="block text-sm font-medium text-gray-700">Email</label>
        <input type="email" id="email" name="email" required class="w-full px-4 py-2 mt-2 border rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500">
      </div>

      <div>
        <label for="password" class="block text-sm font-medium text-gray-700">Password</label>
        <input type="password" id="password" name="password" required class="w-full px-4 py-2 mt-2 border rounded-md shadow-sm focus:outline-none focus:ring-2 focus:ring-blue-500">
      </div>

      <div>
        <label for="confirm_password" class="block text-sm font-medium text-gray-700">Confirm Password</label>

```

```
<input type="password" id="confirm_password" name="confirm_password" required
class="w-full px-4 py-2 mt-2 border rounded-md shadow-sm focus:outline-none focus:ring-2
focus:ring-blue-500">
```

```
</div>
```

```
<button type="submit" class="w-full bg-blue-600 text-white py-2 rounded-md
hover:bg-blue-700 focus:outline-none focus:ring-2 focus:ring-blue-500">Register</button>
```

```
<div class="text-center mt-4">
```

```
<p class="text-sm text-gray-600">Already have an account? <a href="{% url 'login' %}"
class="text-blue-600 hover:underline">Login here</a>.</p>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

Login.html

```
<!-- question/templates/login.html -->
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Login</title>
```

```
<style>
```

```
body {
```

```
background: #f4f4f9;
```

```
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```

display: flex;
justify-content: center;
align-items: center;
height: 100vh;
margin: 0;
}
.login-container {
background: white;
padding: 2rem 3rem;
border-radius: 12px;
box-shadow: 0 4px 20px rgba(0,0,0,0.1);
width: 100%;
max-width: 400px;
}
h2 {
text-align: center;
margin-bottom: 1.5rem;
color: #333;
}
form p {
margin-bottom: 1rem;
}
input[type="text"],
input[type="password"] {
width: 100%;
padding: 10px;
border-radius: 6px;
border: 1px solid #ccc;
}
button {
width: 100%;
padding: 10px;
background-color: #4f46e5;

```

```

        color: white;
        border: none;
        border-radius: 6px;
        font-weight: bold;
        cursor: pointer;
    }
    button:hover {
        background-color: #4338ca;
    }
    .register-link {
        text-align: center;
        margin-top: 1rem;
    }
    .register-link a {
        color: #4f46e5;
        text-decoration: none;
    }
</style>
</head>
<body>
    <div class="login-container">
        <h2>Login</h2>
        <form method="post">
            {% csrf_token %}
            {{ form.as_p }}
            <button type="submit">Login</button>
        </form>
        <div class="register-link">
            <p>Don't have an account? <a href="{% url 'register' %}">Register</a></p>
        </div>
    </div>
</body>
</html>

```


settings.py

```
# question/settings.py
```

```
TEMPLATES = [  
    {  
        ...  
        'DIRS': [BASE_DIR / 'question' / 'templates'],  
        ...  
    },  
]
```

OUTPUT:

The image displays two side-by-side web forms. The left form, titled 'Create an Account', features four input fields labeled 'Username', 'Email', 'Password', and 'Confirm Password', followed by a blue 'Register' button and a link 'Already have an account? Login here.' The right form, titled 'Login', features two input fields labeled 'Username:' and 'Password:', followed by a blue 'Login' button and a link 'Don't have an account? Register'.

Create an Account

Username

Email

Password

Confirm Password

[Register](#)

Already have an account? [Login here.](#)

Login

Username:

Password:

[Login](#)

Don't have an account? [Register](#)

2.

AIM: Create an application in Django to send emails using email settings and Grid Layout

1. **Go to App Passwords Page:**

- Visit: <https://myaccount.google.com/apppasswords>
- Make sure you're logged into the **same Gmail account** where 2FA is enabled.

2. **You'll see two dropdowns:**

- In **"Select App"**, choose Mail.
- In **"Select Device"**, choose Other (Custom name) and give it a name like:
→ Django Mailer (or any name you like)

3. **Click "Generate"**

Google will give you a **16-character password** that looks something like:
abcd efgh ijkl mnop

4. **Copy that password — EMAIL_HOST_PASSWORD for this one in settings.py** you'll paste into your Django settings.

settings.py

```
ALLOWED_HOSTS = [  
    '*',  
    'localhost',  
    '127.0.0.1',  
    '.codeengineprod.blackbucks.me',  
    '3000-aed749c67-f499-4582-aa5f-b184a87ea1a7.codeengineprod.blackbucks.me',  
]
```

Templates

```

'DIRS': [BASE_DIR / 'question' / 'templates'],

CSRF_TRUSTED_ORIGINS = [
    'https://3000-aed749c67-f499-4582-aa5f-b184a87ea1a7.codeengineprod.blackbucks.me',
    'https://*.codeengineprod.blackbucks.me',
    'https://taptapdev.blackbucks.me'
]

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'yourmail@gmail.com'
EMAIL_HOST_PASSWORD = 'abcd efgh ijkl mnop' # Use Gmail App Password

```

templates/send_email.html

```

<!DOCTYPE html>
<html>
<head>
  <title>Send Email</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      display: grid;
      place-items: center;
    }
  </style>

```

```

    height: 100vh;
    background: #f0f2f5;
    margin: 0;
}
.box {
    background: white;
    padding: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    display: grid;
    gap: 1rem;
    width: 300px;
}
input, textarea, button {
    padding: 10px;
    font-size: 1rem;
    width: 100%;
}
button {
    background: #4CAF50;
    color: white;
    border: none;
    cursor: pointer;
}
.success {
    color: green;
    text-align: center;
}
</style>
</head>

```

```

<body>
  <div class="box">
    <h2>Send an Email</h2>
    {% if sent %}
      <p class="success">Email sent successfully!</p>
    {% endif %}
    <form method="post">
      {% csrf_token %}
      {{ form.as_p }}
      <button type="submit">Send</button>
    </form>
  </div>
</body>
</html>

```

urls.py

```

from django.contrib import admin
from django.urls import path
from .views import send_email_view

```

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', send_email_view, name='send_email'),
]

```

views.py

```

from django.shortcuts import render
from django.core.mail import send_mail

```

```
from django import forms
```

```
class EmailForm(forms.Form):
```

```
    recipient = forms.EmailField()
```

```
    subject = forms.CharField(max_length=100)
```

```
    message = forms.CharField(widget=forms.Textarea)
```

```
def send_email_view(request):
```

```
    sent = False
```

```
    form = EmailForm(request.POST or None)
```

```
    if request.method == 'POST' and form.is_valid():
```

```
        send_mail(
```

```
            form.cleaned_data['subject'],
```

```
            form.cleaned_data['message'],
```

```
            'your_email@gmail.com',
```

```
            [form.cleaned_data['recipient']]
```

```
        )
```

```
        sent = True
```

```
    return render(request, 'send_email.html', {'form': form, 'sent': sent})
```

OUTPUT :

Send an Email

Recipient:

Subject:

Message:

Send

WEEK 7 - REGISTRATION AND FORMS

1.

AIM: Create an application in Django using page registration / authentication with Login and Logout Functionality

CODE:

urls.py

```
from django.contrib import admin
from django.urls import path
from django.shortcuts import redirect
from . import views

urlpatterns = [
    path("", lambda request: redirect('dashboard') if request.user.is_authenticated else
    redirect('login')),
    path('admin/', admin.site.urls),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
]
```


views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib import messages

def login_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
            return redirect('dashboard')
        else:
            messages.error(request, 'Invalid username or password.')
    return render(request, 'login.html')

def logout_view(request):
    logout(request)
    return redirect('login')

@login_required(login_url='login')
def dashboard(request):
    return render(request, 'dashboard.html')
```

templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head><title>Dashboard</title></head>
<body>
  <h1>Welcome, {{ request.user.username }}!</h1>
  <a href="{% url 'logout' %}">Logout</a>
</body>
</html>
```

templates/login.html

```
<!DOCTYPE html>
<html>
<head><title>Login</title></head>
<body>
  <h2>Login</h2>
  {% if messages %}
    {% for message in messages %}
      <p style="color:red;">{{ message }}</p>
    {% endfor %}
  {% endif %}
  <form method="POST">
    {% csrf_token %}
    <input type="text" name="username" placeholder="Username" required><br>
    <input type="password" name="password" placeholder="Password" required><br>
    <button type="submit">Login</button>
  </form>
```

</body>

</html>

settings.py

```
ALLOWED_HOSTS = ['*',  
'3000-a508d39b3-13ee-42a6-80dc-1b1369b73355.codeengineprod.blackbucks.me',  
]
```

```
TEMPLATES = [  
    'DIRS': [BASE_DIR / 'question' / 'templates'],  
]
```

```
CSRF_TRUSTED_ORIGINS = [  
    'https://*.codeengineprod.blackbucks.me',  
    'https://taptapdev.blackbucks.me',  
    'https://3000-a508d39b3-13ee-42a6-80dc-1b1369b73355.codeengineprod.blackbucks.me'  
]
```

Run commands in Bash terminal

python manage.py makemigrations

python manage.py migrate

python manage.py createsuperuser - give username, email, password

OUTPUT:

Login

2.

AIM: Create a sample form using Django Forms.

urls.py

```
# question/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('', views.contact_view, name='home'), # This makes '/' go to contact form
    path('contact/', views.contact_view, name='contact'),
]
```

forms.py

```
# question/forms.py

from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100, label="Your Name")
    email = forms.EmailField(label="Your Email")
```

views.py

```
# question/views.py

from django.shortcuts import render
from .forms import ContactForm
```

```

def contact_view(request):
    submitted = False
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            # You can process form.cleaned_data here
            submitted = True
    else:
        form = ContactForm()

    return render(request, 'contact.html', {'form': form, 'submitted': submitted})

```

```

def home_view(request):
    return render(request, 'home.html')

```

templates/contact.html

```

<!-- question/templates/question/contact.html -->

<!DOCTYPE html>
<html>
<head>
    <title>Contact Form</title>
</head>
<body>
    <h2>Contact Form</h2>

```

```

{% if submitted %}

    <p>Thanks for submitting the form!</p>

{% else %}

    <form method="post">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Submit</button>

    </form>

{% endif %}

</body>

</html>

```

home.html

```

<!-- question/templates/question/home.html -->

<h2>Welcome to the Home Page!</h2>

<a href="{% url 'contact' %}">Go to Contact Form</a>

```

settings.py

```

ALLOWED_HOSTS = ['*',
'3000-a54cfbc16-662f-4c39-b7fd-8e5cf402091b.codeengineprod.blackbucks.me',]

TEMPLATES = [
    'DIRS': [BASE_DIR/'question/'/'templates'],
]

CSRF_TRUSTED_ORIGINS = [
    'https://*.codeengineprod.blackbucks.me',
    'https://taptapdev.blackbucks.me',
]

```

```
'https://3000-a54cfbc16-662f-4c39-b7fd-8e5cf402091b.codeengineprod.blackbucks.me'  
]
```

OUTPUT :

Contact Form

Your Name:

Your Email:

WEEK 8 - DATABASE AND CRUD

1.

AIM : Create an app in Django which fetches data from database and show as list and also save objects in database

models.py :

```
# question/models.py
```

```
from django.db import models
```

```
class Item(models.Model):
```

```
    name = models.CharField(max_length=100)
```

```
    description = models.TextField()
```

```
    def __str__(self):
```

```
        return self.name
```

```
class Meta:
```

```
    app_label = 'question'
```

views.py:

```
from django.shortcuts import render, redirect
```

```
from .models import Item
```

```
def item_list(request):
```

```

items = Item.objects.all()

return render(request, 'item_list.html', {'items': items})

def add_item(request):
    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description')
        Item.objects.create(name=name, description=description)
        return redirect('item_list')
    return render(request, 'add_item.html')

```

urls.py :

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.item_list, name='item_list'),
    path('add/', views.add_item, name='add_item'),
]

```

Templates

Create a folder templates/question/

add_item.html

```

<!DOCTYPE html>

<html>

```

```

<head><title>Add Item</title></head>
<body>
  <h2>Add New Item</h2>
  <form method="post">
    {% csrf_token %}
    <input type="text" name="name" placeholder="Name" required><br><br>
    <textarea name="description" placeholder="Description" required></textarea><br><br>
    <button type="submit">Add</button>
  </form>
  <a href="{% url 'item_list' %}">Back to list</a>
</body>
</html>

```

item_list.html

```

<!DOCTYPE html>
<html>
<head><title>Item List</title></head>
<body>
  <h2>Items</h2>
  <ul>
    {% for item in items %}
      <li><strong>{{ item.name }}</strong>: {{ item.description }}</li>
    {% empty %}
      <li>No items yet.</li>
    {% endfor %}
  </ul>
  <a href="{% url 'add_item' %}">Add new item</a>
</body>

```

</html>

Settings.py

```
TEMPLATES = [  
    {  
  
        'DIRS': [BASE_DIR, 'question/templates'], }  
  
    ]  
  
INSTALLED_APPS = [  
    'question',  
]  
  
CSRF_TRUSTED_ORIGINS = [  
    'https://3000-ade195d37-a787-41bc-b79d-251846c64147.codeengineprod.blackbucks.me',  
]
```

Commands to run in terminal:

```
python manage.py makemigrations question  
python manage.py migrate  
python manage.py shell  
>>from question.models import Item  
>>Item.objects.create(name="First Item", description="This is a test item.")  
>>exit()
```

OUTPUT :

Add New Item

[Back to list](#)

Items

- **First Item:** This is a test item
- **fd:** cdf
- **adsfgh:** awdsdc
- **Sush:** dfg

[Add new item](#)

2.

AIM: Create an app in Django for performing CRUD operations on records in a database

models.py

```
from django.db import models

class Record(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    message = models.TextField()

    def __str__(self):
        return self.name
```

forms.py

```
from django import forms
from .models import Record

class RecordForm(forms.ModelForm):
    class Meta:
        model = Record
        fields = ['name', 'email', 'message']
```

urls.py

```
from django.contrib import admin
from django.urls import path
from question import views # import views from your question app

urlpatterns = [
    path('admin/', admin.site.urls),
```

```

path('records/', views.record_list, name='record_list'),
path('records/new/', views.record_create, name='record_create'),
path('records/edit/<int:pk>', views.record_update, name='record_update'),
path('records/delete/<int:pk>', views.record_delete, name='record_delete'),
]

```

views.py

```

from django.shortcuts import render, redirect, get_object_or_404
from .models import Record
from .forms import RecordForm

def record_list(request):
    records = Record.objects.all()
    return render(request, 'question/record_list.html', {'records': records})

def record_create(request):
    form = RecordForm(request.POST or None)
    if form.is_valid():
        form.save()
        return redirect('record_list')
    return render(request, 'question/record_form.html', {'form': form})

def record_update(request, pk):
    record = get_object_or_404(Record, pk=pk)
    form = RecordForm(request.POST or None, instance=record)
    if form.is_valid():
        form.save()
        return redirect('record_list')
    return render(request, 'question/record_form.html', {'form': form})

```

```
def record_delete(request, pk):
    record = get_object_or_404(Record, pk=pk)
    if request.method == 'POST':
        record.delete()
        return redirect('record_list')
    return render(request, 'question/record_confirm_delete.html', {'record': record})
```

Settings.py

```
TEMPLATES = [
    {

        'DIRS': [BASE_DIR, 'question/templates'], }

INSTALLED_APPS = [
    'question',
}

CSRF_TRUSTED_ORIGINS = [
    'https://3000-ade195d37-a787-41bc-b79d-251846c64147.codeengineprod.blackbucks.me',
]
```

question/templates/question/

record_list.html

```
<h1>Records</h1>
<a href="{% url 'record_create' %}">+ Add New Record</a>
<ul>
    {% for record in records %}
        <li>
```



```

    {{ record.name }} | {{ record.email }}
    <a href="{% url 'record_update' record.pk %}">Edit</a>
    <a href="{% url 'record_delete' record.pk %}">Delete</a>
  </li>
{% endfor %}
</ul>

```

record_form.html

```

<h2>Record Form</h2>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Save</button>
</form>

```

record_confirm_delete.html

```

<h2>Confirm Delete</h2>
<p>Are you sure you want to delete "{{ record.name }}"?</p>
<form method="post">
  {% csrf_token %}
  <button type="submit">Yes, Delete</button>
</form>

```

Run the Server

```

python manage.py makemigrations question
python manage.py migrate

```

OUTPUT:

Record Form

Name:

Email:

Message: